

Rhythm Combat - Spatial Lane Layer

Data: 4 maggio 2026

Obiettivo

Implementare il layer richiesto per il sistema rhythm-combat senza riscrivere il progetto e senza modificare la pipeline di import StepMania già esistente.

La richiesta era aggiungere supporto a:

- ? layout spaziale delle 6 lane, numerate da 0 a 5;
- ? movimento delle note dal centro del campo verso il centro della lane;
- ? continuazione della nota oltre la lane se non viene premuta;
- ? judgment basato sulla distanza dal centro della lane;
- ? tolleranze configurabili;
- ? input per lane configurabile da editor e modificabile a runtime;
- ? integrazione pulita con `CombatRhythmEngine``.

Vincoli rispettati

- ? Non è stata riscritta la pipeline `.sm``.
- ? Non sono stati modificati `SMParse``, `MeasureUtils``, `RowGenerator``, `FrameGenerator``, `ChartDataAsset`` o `ChartRow``.
- ? Il nuovo core è stato tenuto separato da `UnityEngine`` quando possibile.
- ? Non sono stati usati `record``, `record struct``, `init`` o file-scoped namespace.
- ? Il refactor di `CombatRhythmEngine`` è stato minimo e retrocompatibile.
- ? `JudgmentService`` resta disponibile per score config e hold release validation.

Layout delle lane

Il nuovo layout standard rappresenta il campo così:

```
0 | 1 | 2
-----
3 | 4 | 5
```

Le posizioni standard sono:

```
lane 0 = (-2, 1)
lane 1 = ( 0, 1)
lane 2 = ( 2, 1)
lane 3 = (-2, -1)
lane 4 = ( 0, -1)
lane 5 = ( 2, -1)
spawn center = (0, 0)
```

Ogni lane ha un `ToleranceRadius`, usato per normalizzare la distanza della nota dal centro della lane.

File aggiunti

Core geometry

```
? `Assets/Scripts/RhythmCombat/Domain/Geometry/Double2.cs`  
? `Assets/Scripts/RhythmCombat/Domain/Geometry/LaneGeometry.cs`  
? `Assets/Scripts/RhythmCombat/Domain/Geometry/BattlefieldLayout.cs`
```

`Double2` è una struct pura C# per coordinate 2D. Espone operazioni base come `Lerp`, `Distance`, `Add`, `Subtract` e `Multiply`.

`LaneGeometry` descrive una lane fisica tramite:

```
? `LaneIndex`  
? `Center`  
? `ToleranceRadius`
```

`BattlefieldLayout` contiene lo `SpawnCenter` e la mappa `laneIndex` -> `LaneGeometry`. Include una factory `CreateStandard( . . . )` per il layout 3 sopra e 3 sotto.

Core movement

```
? `Assets/Scripts/RhythmCombat/Domain/Movement/NoteTravelSettings.cs`  
? `Assets/Scripts/RhythmCombat/Domain/Movement/NotePositionResult.cs`  
? `Assets/Scripts/RhythmCombat/Domain/Movement/NoteMotionService.cs`
```

`NoteTravelSettings` configura:

```
? `ApproachDurationSeconds`  
? `DespawnAfterHitSeconds`
```

`NoteMotionService` calcola la posizione della nota nel tempo.

Formula:

```
spawnTime = note.HitTimeSeconds - ApproachDurationSeconds  
progress = (currentTimeSeconds - spawnTime) / ApproachDurationSeconds  
position = LerpUnclamped(SpawnCenter, LaneCenter, progress)
```

Quando `progress == 1`, la nota si trova esattamente al centro della lane al tempo dettato dalla chart.

Quando `progress > 1`, la nota continua oltre il centro della lane nella stessa direzione.

`NotePositionResult` restituisce:

```
? `Position`  
? `NormalizedProgress`
```

? `ShouldDespawn`

Core judgment

? `Assets/Scripts/RhythmCombat/Domain/Timing/INoteJudgmentService.cs`
? `Assets/Scripts/RhythmCombat/Domain/Timing/SpatialJudgmentConfig.cs`
? `Assets/Scripts/RhythmCombat/Domain/Timing/SpatialJudgmentService.cs`
? `Assets/Scripts/RhythmCombat/Domain/Timing/TemporalNoteJudgmentService.cs`

`INoteJudgmentService` astrae il judgment della nota:

```
JudgmentResult Evaluate(CombatNote note, double currentTimeSeconds);  
bool IsMissed(CombatNote note, double currentTimeSeconds);
```

Questo permette di usare sia judgment temporale sia judgment spaziale.

`SpatialJudgmentConfig` definisce le soglie percentuali:

```
PerfectPercent  
GoodPercent  
BadPercent
```

Esempio:

```
Perfect <= 0.10  
Good    <= 0.25  
Bad     <= 0.50  
Miss    > 0.50
```

`SpatialJudgmentService`:

1. calcola la posizione della nota tramite ``NoteMotionService``;
2. recupera il centro della lane da ``BattlefieldLayout``;
3. calcola la distanza tra nota e centro lane;
4. normalizza la distanza dividendo per ``ToleranceRadius``;
5. assegna ``Perfect``, ``Good``, ``Bad`` o ``Miss``.

Il delta temporale resta salvato dentro `JudgmentResult` come:

```
currentTimeSeconds - note.HitTimeSeconds
```

`TemporalNoteJudgmentService` mantiene il comportamento precedente usando internamente `JudgmentService`.

File Unity aggiunti

? `Assets/Scripts/RhythmCombat/Unity/LaneKeyBinding.cs`
? `Assets/Scripts/RhythmCombat/Unity/LaneInputPreset.cs`
? `Assets/Scripts/RhythmCombat/Unity/LaneInputReader.cs`
? `Assets/Scripts/RhythmCombat/Unity/SpatialRhythmCombatDebugHarness.cs`

`LaneKeyBinding` è una classe serializzabile Unity con:

```
? `laneIndex`  
? `KeyCode key`
```

`LaneInputPreset` è uno `ScriptableObject` configurabile da inspector. Permette preset come:

```
lane 0 -> A  
lane 1 -> S  
lane 2 -> D  
lane 3 -> J  
lane 4 -> K  
lane 5 -> L
```

`LaneInputReader` legge il preset e permette:

```
? `ApplyPreset(LaneInputPreset preset)`  
? `SetBinding(int laneIndex, KeyCode key)`  
? `GetLaneDown(int laneIndex)`  
? `GetLaneUp(int laneIndex)`
```

Questo consente di cambiare configurazione anche a runtime.

Modifica a `CombatRhythmEngine`

File modificato:

```
? `Assets/Scripts/RhythmCombat/Runtime/CombatRhythmEngine.cs`
```

È stato introdotto il campo:

```
private readonly INoteJudgmentService _noteJudgmentService;
```

Il vecchio costruttore è stato mantenuto:

```
CombatRhythmEngine(  
    JudgmentService judgmentService,  
    SuperMeterService superMeterService,  
    DamageResolver damageResolver,  
    DefenseResolver defenseResolver,  
    SuperModeService superModeService)
```

Questo costruttore usa automaticamente:

```
new TemporalNoteJudgmentService(judgmentService)
```

Quindi il comportamento esistente resta invariato.

È stato aggiunto un nuovo costruttore:

```
CombatRhythmEngine(  
    INoteJudgmentService noteJudgmentService,  
    JudgmentService judgmentService,  
    SuperMeterService superMeterService,  
    DamageResolver damageResolver,  
    DefenseResolver defenseResolver,
```

SuperModeService superModeService)

Questo permette di iniettare `SpatialJudgmentService`.

Modifiche logiche:

- ? ``ProcessTapHit`` ora usa ``_noteJudgmentService.Evaluate(...)``.
- ? I miss automatici usano ``_noteJudgmentService.IsMissed(...)``.
- ? ``JudgmentService`` resta usato per ``Config.GetScore(...)``, hold release validation e hold release timeout.

Sono stati preservati:

- ? score su hit valido;
- ? incremento multiplier;
- ? reset multiplier su miss;
- ? caricamento super su hit;
- ? danno su attack note;
- ? danno al party su defense miss;
- ? gestione hold start/release.

Debug harness

File:

- ? ``Assets/Scripts/RhythmCombat/Unity/SpatialRhythmCombatDebugHarness.cs``

Il debug harness crea:

- ? ``BattlefieldLayout`` standard;
- ? ``NoteTravelSettings``;
- ? ``SpatialJudgmentConfig``;
- ? ``SpatialJudgmentService``;
- ? una ``CombatChart`` fake con:
 - una tap attack;
 - una tap defense;
 - una hold note;
- ? party, enemy e super meter fake;
- ? un ``CombatRhythmEngine`` con judgment spaziale.

Stampa via `Debug . Log`:

- ? posizione nota;
- ? progress;
- ? distanza dal centro lane;
- ? distanza normalizzata;

- ? judgment;
- ? score;
- ? multiplier;
- ? super gained;
- ? danno applicato.

Come testare

1. Aprire Unity.
2. Creare un GameObject vuoto in una scena di test.
3. Aggiungere il componente `SpatialRhythmCombatDebugHarness`.
4. Abilitare `Run On Start`, oppure usare il context menu `Run Spatial Rhythm Combat Harness`.
5. Avviare la scena.
6. Verificare i log in console.

Per verificare solo la compilazione da terminale:

```
dotnet build Outcasts.sln -m:1
```

La build e stata eseguita con successo:

```
Compilazione completata.  
Avvisi: 0  
Errori: 0
```

Assunzioni

- ? Il layout standard usa coordinate logiche, non ancora coordinate finali di UI o world-space.
- ? La conversione da `Double2` a `Vector2` o `Vector3` sara gestita dal layer visual.
- ? `LaneInputReader` usa `KeyCode` per un primo layer editor/runtime semplice. Potra essere adattato al nuovo Input System se necessario.
- ? La miss logica e il despawn visual sono separati: `IsMissed(...)` puo marcare la nota come miss, mentre `ShouldDespawn` indica quando puo essere rimossa visivamente.
- ? Il vecchio behavior temporale resta disponibile tramite `TemporalNoteJudgmentService`.

Stato finale

Il layer richiesto e stato aggiunto senza riscrivere i sistemi esistenti.

La nota ora puo:

- ? partire dal centro;
- ? arrivare al centro della lane al tempo della chart;
- ? proseguire oltre la lane se non premuta;
- ? essere giudicata in base alla distanza dal centro con tolleranza configurabile;

? usare input per lane configurabile da editor e runtime.

Il tutto è agganciabile a `CombatRhythmEngine` tramite `INoteJudgmentService`.